

Architectural Leakage as Information Flow in Modern SoCs

Sadat Muhtasim Kabir
University of New South Wales
Sydney, Australia
z5443562@ad.unsw.edu.au

Abstract—This research proposal introduces a graph-based framework for reasoning about architectural leakage independent of specific implementations. By modelling leakage as hidden-to-observable information-flow paths, the approach classifies leakage-capable interactions and provides a method for testing interfaces for previously unknown leakage paths in heterogeneous SoCs.

Index Terms—Side-channel attacks, SoC security.

I. BACKGROUND AND RESEARCH GAP

Modern processors and SoCs rely on architectural abstractions that are assumed to expose only intended program-visible state. Hardware verification therefore focuses on functional correctness. However, attacks such as $\text{\AA}$ PIC Leak [1], Spectre [2], and Meltdown [3] show that this assumption does not hold: even when architectural behaviour is correct, microarchitectural state can still influence observable outputs through stale data, speculation, timing variation, or shared resources.

This has led to a wide range of analysis techniques, but they tend to be tied to specific mechanisms or abstraction levels. Spectre and Meltdown focus on transient execution, cache attacks on timing variation, leakage templates on recurring patterns [4], and tools such as Revizor on empirical detection [5]. Formal approaches like leakage contracts, GLIFT, and SecVerilog enforce information-flow constraints, but only within fixed abstraction boundaries [6]–[8].

At a higher level, prior survey work frames leakage as a correlation chain—secret data influences runtime behaviour, which then propagates to observable microarchitectural effects such as cache timing, TLB activity, DRAM row-buffer state, branch prediction, or SMT contention [9]. While this perspective unifies many known attacks, it still operates retrospectively: it explains how leakage occurs once a mechanism is identified, but does not help determine whether an *arbitrary* interface is inherently capable of leaking.

This gap becomes more pronounced as SoCs integrate increasingly heterogeneous components—CPUs, GPUs, accelerators, DMA engines, memory controllers, and peripherals—where interactions scale combinatorially with system complexity. What is missing is a system-level, implementation-agnostic framework that can reason about leakage potential before specific attack mechanisms are known.

II. OBJECTIVES

I propose the following research questions:

RQ1: Can hidden-to-observable paths determine whether an architectural interface is leak-capable independent of a specific mechanism?

RQ2: What structural properties—state dependence, persistence, sharing, observability, incomplete initialisation—govern the capacity to induce such paths?

RQ3: Can diverse leakage mechanisms (cache timing, speculative execution, stale-state exposure, SMT contention, DMA-based access, fault-induced deviations) be unified as instances of a graph-based abstraction?

RQ4: Can the graph model support both formal (SMT-based) and probabilistic leakage reasoning?

III. RESEARCH METHODOLOGY

The system can be modelled as $F : S_h \times \Omega \rightarrow S_o$, where S_h is hidden state (e.g secret keys, cache), Ω represents non-secret inputs, and S_o is observable output (e.g timing, latency, output). Deterministic leakage exists if $\exists s_1 \neq s_2 \in S_h, \exists \omega \in \Omega$ such that $F(s_1, \omega) \neq F(s_2, \omega)$. The model extends to probabilistic leakage where $P(S_o | S_h=s_1) \neq P(S_o | S_h=s_2)$, capturing noisy channels where hidden states produce statistically distinguishable but not necessarily distinct outputs. Then in a directed graph representation, $G = (V, E)$, nodes represent architectural, microarchitectural, or physical states and edges represent influence. Leakage corresponds to an unintended path $S_h \rightarrow \dots \rightarrow S_o$ absent from the architectural specification. Edges capture data

flow, timing variation, contention, residual-state propagation, and fault effects. The abstraction generalises prior formulations of leakage as a correlation chain between secret state, runtime behaviour, and observable signals [9]. In this model, each stage corresponds to nodes and edges in G . For example, cache timing attacks follow $S_h \rightarrow$ access pattern $\rightarrow$ cache state $\rightarrow$ timing, while DMA leakage follows $S_h \rightarrow$ memory $\rightarrow$ DMA engine $\rightarrow$ device buffer.

The methodology proceeds in three phases. **Phase 1:** Representative platforms—open RISC-V cores, cycle-level simulators, microcontroller-class SoCs—are selected and interfaces (memory-mapped registers, DMA engines, accelerators, interrupt controllers, peripherals) are mapped into G by identifying hidden states, intermediate resources, and observable outputs. **Phase 2:** Controlled experiments compare executions differing only in S_h . Prior state is manipulated to embed identifiable patterns, testing residual-state exposure across context switches, privilege transitions, and DMA transfers. Randomised input generation spans instruction sequences, memory access patterns, and peripheral transactions. Observations include returned values, timing, exception behaviour, memory latency, and bus-visible effects. **Phase 3:** Operations are classified by structural properties (persistence, sharing, state dependence, observability). Known mechanisms are mapped onto G to test whether they emerge as hidden-to-observable paths, validating the model’s generality.

Deterministic leakage detection can be formulated as a satisfiability query over S_h and Ω , checking whether there exist two distinct hidden states that produce different observable outputs under the same conditions. If such an assignment exists, the solver returns a counterexample corresponding to a leakage-inducing execution; otherwise, the interface satisfies non-interference. Probabilistic leakage is instead characterised by comparing output distributions, using statistical divergence and mutual information to detect distinguishability. Together, these formulations connect structural leakage paths to both formal verification and empirical detection, enabling the framework to reason about architectural leakage in a mechanism-agnostic manner across diverse system components.

IV. EVALUATION

The framework is evaluated against five criteria. **Detection:** Observable deviations correlated with S_h —both deterministic (differing returned values) and probabilistic (distinguishable timing distributions)—should be identified across tested interfaces. **Generalisation:** Multiple leakage classes (cache timing, speculative execution, stale-state, SMT contention, DMA, fault-based) should be expressible as structurally equivalent hidden-to-observable paths without mechanism-specific assumptions. **Predictive value:** Operations classified as high-risk by the model should empirically exhibit stronger leakage signals than low-risk operations, validating the graph classification as a triage tool. **Consistency:** Leakage signals should remain statistically stable under varied inputs, quantified via variance, mutual information, or distributional divergence. **Coverage:** All evaluated leakage mechanisms should be captured without modifying the underlying model, demonstrating that diverse attacks reduce to a common structural form.

REFERENCES

- [1] P. Borrello et al., “ $\text{\AA}$ PIC Leak,” USENIX Security, 2022.
- [2] P. Kocher et al., “Spectre,” IEEE S&P, 2019.
- [3] M. Lipp et al., “Meltdown,” USENIX Security, 2018.
- [4] A. Ibrahim et al., “Leakage templates,” ACM CCS, 2022.
- [5] J. Oleksenko et al., “Revizor,” ASPLOS, 2022.
- [6] Z. Wang et al., “Leakage contracts,” IEEE S&P, 2023.
- [7] M. Tiwari et al., “GLIFT,” DAC, 2009.
- [8] D. Zhang et al., “SecVerilog,” ASPLOS, 2015.
- [9] X. Lou et al., “Survey of microarchitectural side channels,” ACM CSUR, 2021.